

Public Mobile App for Arogya-SMC: Complete Design Document

Overview

The **Arogya-SMC Citizen App** is a public-facing mobile application that provides Solapur residents with real-time access to hospital availability, public health advisories, and outbreak alerts. The app is designed to be lightweight, intuitive, and accessible to users with varying levels of digital literacy.

1. Technology Stack (100% Free)

Core Framework

Component	Technology	Justification	Cost
Framework	Flutter 3.27+	Single codebase for Android & iOS; rich widget library; excellent documentation	Free (Open source)
Language	Dart 3.5+	Modern, type-safe, excellent Flutter integration	Free
IDE	VS Code	Lightweight, excellent Flutter plugins, free	Free

State Management & Architecture

Following Flutter's official MVVM architecture recommendations :

Component	Technology	Purpose
State Management	Riverpod 2.5+	Modern, testable, no context dependency; gold standard for 2026
Navigation	GoRouter 14.0+	Declarative routing with deep linking support
Architecture Pattern	MVVM (Model-View-ViewModel)	Clean separation of concerns

Networking & Data

Component	Technology	Purpose
HTTP Client	Dio 5.4+	Powerful interceptor support, form data, cancellation
Local Storage	Hive 2.2+	Fast, lightweight NoSQL for caching
Secure Storage	flutter_secure_storage	For any tokens (though public app has no auth)

Utilities

Component	Technology	Purpose
Location	geolocator 12.0+	GPS access for ward detection (consent-based)
Permissions	permission_handler	Runtime permission management
Push Notifications	firebase_messaging	Free tier generous for prototype
Localization	easy_localization	Multi-language support (Marathi, Hindi, English)
Image Loading	cached_network_image	Efficient image caching
Pull to Refresh	flutter_native_refresh	Native refresh indicator

Development Tools

Tool	Purpose
Flutter DevTools	Performance profiling, widget inspector
FVM (Flutter Version Manager)	Manage Flutter SDK versions
Mason	Template generation for consistent architecture

2. Features & Functions

Core Features (Must-Have)

Feature	Description	Priority
F1: Ward Detection	Auto-detect ward via GPS (with consent) or manual dropdown selection	High
F2: Facility Lookup	Display list of nearby hospitals with real-time bed availability	High
F3: Bed Status Display	Show available beds, ICU, ventilators, oxygen status with color coding	High
F4: Facility Details	Tap to view hospital contact, address, directions	Medium
F5: Advisories Feed	List of public health advisories from SMC	High
F6: Outbreak Alerts	Receive push notifications for ward-specific alerts	Medium
F7: Offline Cache	Store last viewed facility data for offline access	Medium
F8: Multi-language	Marathi, Hindi, English interface	High
F9: Simple Navigation	Bottom navigation bar with 3-4 main sections	High

Data Requirements per Feature

Feature	Data Needed	API Endpoint
---------	-------------	--------------

Feature	Data Needed	API Endpoint
F1	Ward list	/api/public/wards (static)
F2, F3	Facilities with capacity	/api/public/facilities?ward={code}
F4	Facility contact details	/api/public/facilities/{id}
F5	Advisories list	/api/public/advisories
F6	Alert subscription via FCM	FCM topic per ward

3. Pages / Screens Design

Screen Overview

Screen	Route	Purpose
Splash Screen	/	Initial loading, ward detection
Ward Selection	/ward-select	Manual ward selection (fallback)
Home / Facilities	/home	Main screen showing facility list
Facility Detail	/facility/{id}	Detailed view of single hospital
Advisories	/advisories	List of all public health advisories
Alert History	/alerts	Past outbreak alerts (optional)
Settings	/settings	Language selection, about

Screen Flow Diagram

```

[ Splash ] → [ Ward Detection ]
      ↓
[ Ward Select? ] ← (if GPS fails/not granted)
      ↓
[ Home / Facilities ] → [ Facility Detail ]
      ↓               ↓
[ Advisories ]       [ Back to Home ]
      ↓
[ Settings ]

```

Navigation Structure:

- Bottom Navigation Bar on main screens: **Home, Advisories, Settings**
- Facility Detail is a drill-down screen (back button to Home)

4. Detailed Screen Specifications

Screen 1: Splash Screen (/)

Purpose:

- Show app logo and loading indicator
- Detect ward via GPS
- Pre-fetch initial data

UI Elements:

- App logo (Arogya-SMC)
- Loading spinner
- "Detecting your location..." text

Logic:

- Request location permission (if not already granted)
- If granted → get coordinates → reverse geocode to ward code → navigate to Home
- If denied → navigate to Ward Selection screen
- If timeout → navigate to Ward Selection

State Management:

- ViewModel tracks location status, ward code
- Uses Geolocator plugin

Screen 2: Ward Selection (/ward-select)

Purpose: Fallback screen when GPS unavailable/denied

UI Elements:

- Dropdown / searchable list of Solapur wards
- "Confirm" button
- "Use GPS" button (retry permission)

Logic:

- Fetch ward list from API (static data)
- On confirm → save selected ward → navigate to Home

Data Model:

```
class Ward {  
    final String code;  
    final String name;  
    final String? localName; // Marathi  
}
```

Screen 3: Home / Facilities (/home)

Purpose: Main screen showing hospitals with bed availability

UI Elements:

- Ward indicator (e.g., "Current Ward: Sadar Bazaar")
- Last updated timestamp
- Facility cards grid/list
- Pull-to-refresh
- Search/filter (optional)

Facility Card Design:

 Civil Hospital
 Beds: 25/100 (25 available)
 ICU: 2/10 (2 available)
 Ventilators: 1/8 (1 available)
 Oxygen: ☒ Available
→ Tap for details

Color Coding:

- Green: >30% available
- Orange: 10-30% available
- Red: <10% available

Logic:

- Fetch facilities for current ward from API
- Refresh every 5 minutes (or pull to refresh)
- Cache response locally (Hive) for offline viewing
- Show loading skeleton while fetching

API Call:

```
Future<List<Facility>> getFacilities(String wardCode)
```

Screen 4: Facility Detail (/facility/{id})

Purpose: Detailed view of single hospital

UI Elements:

- Hospital name, address, contact
- Full capacity metrics in detail
- "Get Directions" button (opens Google Maps)
- "Call" button (tel: link)
- Back button

Logic:

- Fetch facility details by ID
- Show loading indicator
- Handle deep links

Screen 5: Advisories (/advisories)

Purpose: List of public health advisories from SMC

UI Elements:

- List of advisories with:
 - Title
 - Date
 - Brief description
 - Severity tag (Low/Medium/High)
- Tap to expand full content

Logic:

- Fetch advisories from API
- Sort by date (newest first)
- Show empty state if none

Screen 6: Settings (/settings)

Purpose: App settings and info

UI Elements:

- Language selector (Marathi, Hindi, English)
- About section
- App version
- Privacy policy link
- Clear cache option

5. Navigation Flow

Bottom Navigation Bar

The app uses a **persistent bottom navigation bar** for main sections:

Tab	Icon	Screen	Requires Ward?
1		Home (Facilities)	Yes
2		Advisories	No (shows all)
3		Settings	No

Navigation Stack Management

Following Flutter's nested navigation patterns :

- **Top-level Navigator** manages authentication (none needed) and ward selection flow
- **Nested Navigator** within Home manages facility list → detail transitions
- This ensures bottom nav stays persistent while drilling down

Deep Linking Support

Support for:

- `arogyasmc://facility/{id}` – opens facility detail directly
- `arogyasmc://advisories` – opens advisories screen

6. Data Flow Architecture (MVVM)

Following Flutter's official architecture guide :

Layer Structure

```
UI Layer (Views)
  ↓
ViewModels (Riverpod providers)
  ↓
Repositories (Data sources)
  ↓
Services (API clients)
  ↓
External (Backend API)
```

Component Details

1. Services (API Clients)

- `FacilityService` – GET facilities, facility details
- `AdvisoryService` – GET advisories
- `WardService` – GET ward list

2. Repositories

- `FacilityRepository` – fetches from API, caches to Hive
- `AdvisoryRepository` – fetches advisories
- Provides streams/async data to ViewModels

3. ViewModels (Riverpod Providers)

- `facilityListProvider` – loads and filters facilities

- **facilityDetailProvider** – loads single facility
- **advisoryListProvider** – loads advisories
- **wardProvider** – current selected ward
- **settingsProvider** – language preference

4. Views

- Stateless widgets consuming providers
- Handle loading/error/empty states

Offline Caching Strategy

- Facility data cached in Hive with TTL (5 minutes)
- Advisories cached with TTL (1 hour)
- Ward list cached permanently (rarely changes)
- Show cached data immediately, refresh in background

7. Push Notification Flow

Using Firebase Cloud Messaging (FCM):

1. **Initialization:** On first launch, request notification permission
2. **Topic Subscription:** When ward selected, subscribe to **ward_{code}** topic
3. **Receiving:** App receives notifications when SMC issues ward-specific alerts
4. **Handling:** On tap, navigate to relevant screen (advisories or facility)

Free Tier Considerations:

- FCM is completely free
- No limits on number of messages
- Works for prototype scale

8. Implementation Priority (13 Days)

Given the 13-day timeline, focus on:

Priority	Screens/Features	Day Allocation
P1 (Must Have)	Splash, Ward Selection, Home (facility list), API integration, basic caching	Days 1-5
P2 (Should Have)	Facility Detail, Advisories screen, pull-to-refresh	Days 6-8
P3 (Nice to Have)	Multi-language, push notifications, settings	Days 9-11
Buffer	Testing, bug fixes, polish	Days 12-13

Day-by-Day Breakdown

Day	Focus
1	Project setup, architecture, Riverpod providers structure
2	Services & Repositories layer, API integration
3	Splash screen, ward detection logic
4	Home screen UI, facility cards
5	Connect Home to API, display real data
6	Facility Detail screen
7	Advisories screen
8	Pull-to-refresh, error handling, loading states
9	Multi-language setup (Marathi translations)
10	Push notifications (FCM)
11	Settings screen, polish
12	Testing on multiple devices
13	Final bug fixes, video recording

9. Key Success Criteria for Demo

The 4-5 minute video should clearly show:

1. **Ward Detection** – App detects ward (or manual selection)
2. **Facility List** – Hospitals shown with bed availability
3. **Color Coding** – Green/orange/red based on availability
4. **Refresh** – Pull to refresh updates data
5. **Advisories** – List of public health messages
6. **Language Switch** – Change to Marathi (optional but impressive)
7. **Notification** – (If implemented) Show alert received

10. Project Structure

```
lib/  
├─ main.dart  
├─ app.dart  
├─ core/  
│   ├── constants/  
│   ├── themes/  
│   └─ utils/  
└─ data/
```

```

├── models/
│   ├── facility.dart
│   ├── advisory.dart
│   └── ward.dart
├── repositories/
│   ├── facility_repository.dart
│   ├── advisory_repository.dart
│   └── ward_repository.dart
├── services/
│   ├── api_client.dart
│   ├── facility_service.dart
│   └── advisory_service.dart
├── providers/
│   ├── facility_provider.dart
│   ├── advisory_provider.dart
│   └── settings_provider.dart
├── views/
│   ├── splash/
│   │   ├── splash_screen.dart
│   │   └── splash_view_model.dart
│   ├── ward_select/
│   │   ├── ward_select_screen.dart
│   │   └── ward_select_view_model.dart
│   ├── home/
│   │   ├── home_screen.dart
│   │   ├── home_view_model.dart
│   │   └── widgets/
│   │       └── facility_card.dart
│   ├── facility_detail/
│   ├── advisories/
│   └── settings/
└── l10n/ (localization files)

```

This design gives you a complete blueprint for building the public mobile app. All tools and libraries are free, well-documented, and production-ready. The MVVM architecture ensures the app remains maintainable as you add features. Good luck with the implementation!